

Project Created → May 20, 2026

Last update →

Oct 2, 2026 1:30 PM EDT

PROJECT “PACEPAL”

Point of contact → Lucas Camarota

Phone: +13054587593

Email: lucasocamarota@gmail.com

PROJECT “PACEPAL”

02

TABLE OF CONTENTS

“click any link to jump to page.”

- Table of Contents-2 [Project PACEPAL](#)

-

- Introduction-3 [Project PACEPAL](#)

-

- The Concept-5 Project PACEPAL

-

- The Research-6 Project PACEPAL

-

- Programming/Soldering-8 Project PACEPAL

-

PROJECT “PACEPAL”

03

INTRODUCTION

PacePal is the name of my personal project, a compact, handlebar-mounted display designed for ease of use and comfort during everyday cycling activities. PacePal is an engineering project fully designed, programmed, and engineered by me (Lucas Camarota). I will be using articles, blogs, videos, and documentation for the use of learning, and I'll refrain from using AI to make the project as clean and learning-based as possible. I am using LanguageTool to improve my writing while I document

all of this since I'm still in middle school and I would rather not make any mistakes while writing. PacePal will take inspiration from many projects similar to PacePal to improve and brush up on my engineering/programming skills. The purpose of this project is to create a practical, real-world device that provides riders useful information at a glance, without the complexity or cost of commercial alternatives. On a personal level, my goal is to grow as an engineer by gaining hands-on experience in hardware programming, electronics, and the challenge of bringing together multiple disciplines into a single functional build. I also intend to eventually document programming progress and more on GitHub for more open-source use, depending on the direction of the project later on.

PROJECT “PACEPAL”

04

GOALS

- My goal for this project is to fully complete and finish a handlebar-mounted display while optimizing the number of abilities learned and also making it as original as possible without using as many prebuilt parts as necessary.

- I want to gain as much engineering, programming, and circuitry experience as possible. I should be able to use the skills learned from this project, whether finished or not, to apply them to real-world activities/projects.
- Minimize the use of AI as much as possible for a cleaner and better learning-based project. If AI is used, it should exclusively be used for debugging and project-ending purposes.
- PacePal should be as compact and portable as possible and detachable from any handlebar on a bike. It should avoid communicating with or using anything to measure the bike's speed using the wheel of the bike. Additionally, it should all be based inside the container of the electronics, aka the IMU and GPS

In the end, the main goal shouldn't be so focused on finishing this project, but it should be about learning from it.

--

PROJECT “PACEPAL”

05

THE “CONCEPT”

DISCLAIMER: At the time of writing this, the project has just begun; some of these features may not be accurate or may not be included at all. From here on out, some of the information may be outdated, but I intend to show all of it for the purpose of presenting progress.

PacePal is a compact, self-contained cycling device designed to deliver real-time ride data and safety awareness directly to the handlebars of any standard bicycle. The device is built around a combination of a Global Positioning Unit (GPS)

module and an Inertial Measurement Unit (IMU), where the GPS provides accurate location-based speed tracking. The IMU, comprising an accelerometer and gyroscope, supplements that can detect any movement/motion, orientation, and changes in acceleration. Together, they enable various features, including a live speedometer, crash detection, lean angle monitoring, and road quality sensing, all aimed at making the rider more informed and safe throughout their ride.

On the hardware side of things, PacePal is powered by an Arduino microcontroller, which handles sensor input, display output, and overall device logic. All data is presented through a 3.5-inch screen mounted on a custom 3D-printed enclosure, giving the rider a clear and readable interface at a glance. The build may also include physical buttons for navigating between screens and additional components that will be incorporated as the project develops.

PROJECT “PACEPAL”

06

THE RESEARCH

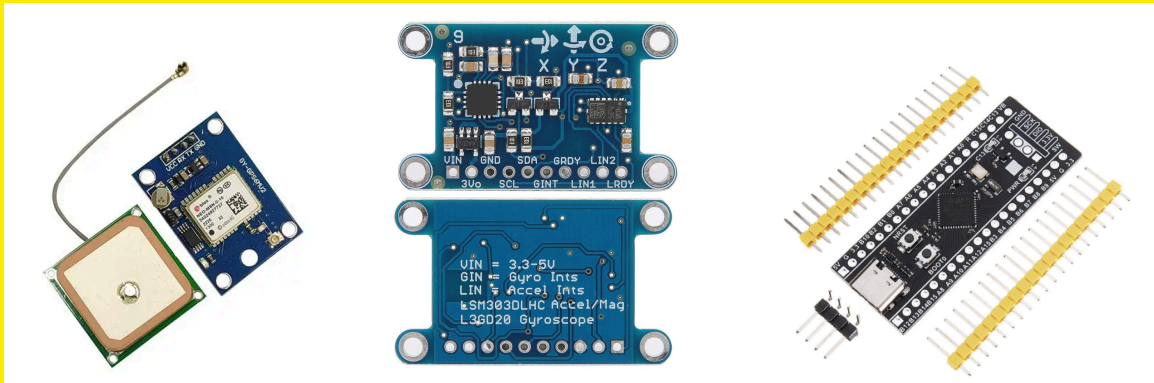
Reasoning:

Research is a tremendously important part of any project, as it informs you of later decision-making, future opportunities, and finding out things you may have never known about your project. According to the article “**Purpose of Research: Understanding Its Role and Importance in Knowledge Building**” by the Coursera staff, they state, “Research can help you make more informed decisions by providing objective data, uncovering hidden factors, validating your strategies, and reducing guesswork. This can lead to more successful, less biased choices based on facts rather than assumptions. By using research to inform your decisions, you can identify trends and assess risks, with the results helping to increase your confidence in your outcome.” Knowing what research is and how important it is to do can really benefit a project.

PARTS

With all of this in mind, I started with the parts. Once I had a concept of what I was doing for this project, I started looking at the parts based on how important they'll be towards the project. First, I looked at the STM32, which will be the microcontroller of this project. A microcontroller (MCU) is essentially a small, self-contained computer built on a single integrated circuit. The microcontroller will be able to control everything inside PacePal while also being able to display the speed, tilt, and more of the bike. Finding the right one wasn't too hard, as I just needed one with a good amount of RAM and pins. Next, I looked at the GPS module (Global Positioning System). To make sure I got the correct module with the right specifications, I did some research on different types of GPS modules. I landed on the NEO series of GPS modules. The NEO series had three different modules: NEO-6M, NEO-7M, and NEO-8M. The NEO-6M was the best budget-friendly option and was also accurate. The NEO-7M landed right in the middle of accuracy and price, and finally, the NEO-8M was the most accurate one, while also being the most expensive out of the three. In the end, I went with the NEO-8M to ensure GPS accuracy. Lastly, I had to find a reliable IMU (Inertial Measurement Unit) module. An IMU can have three components: accelerometers, which measure linear acceleration, gyroscopes, which measure angular velocity, and, while optional, they can have magnetometers that can detect magnetic fields. When all three sensors are combined, the IMU can track movement across nine axes of motion, often referred to as **9 degrees of freedom (9-DOF)**. When looking for an IMU module, I at least wanted one with a gyroscope for tilt and lean and an accelerometer for acceleration detection, even if I might not use it for displaying the speed. Eventually, I found an IMU module that was compatible with an MCU and was surprisingly 11DOF, but they labeled it as 10DOF which was more than enough either way. This was all I needed for now since the rest of the parts were basic items like the power supply, jumper wires, and display. For the moment, I only needed the GPS, IMU, and MCU for programming purposes.

These are example photos of what I ordered:



GPS

IMU

STM32

STM32: WeAct Studio Blackpill v.3.1

GPS: NEO-M8U-0-10

IMU: L3GD20 gyro, LSM303D accel and magno

NOTE: The IMU example photo is NOT accurate; it is a 10DOF



PROJECT “PACEPAL”

08

-Soldering-

—

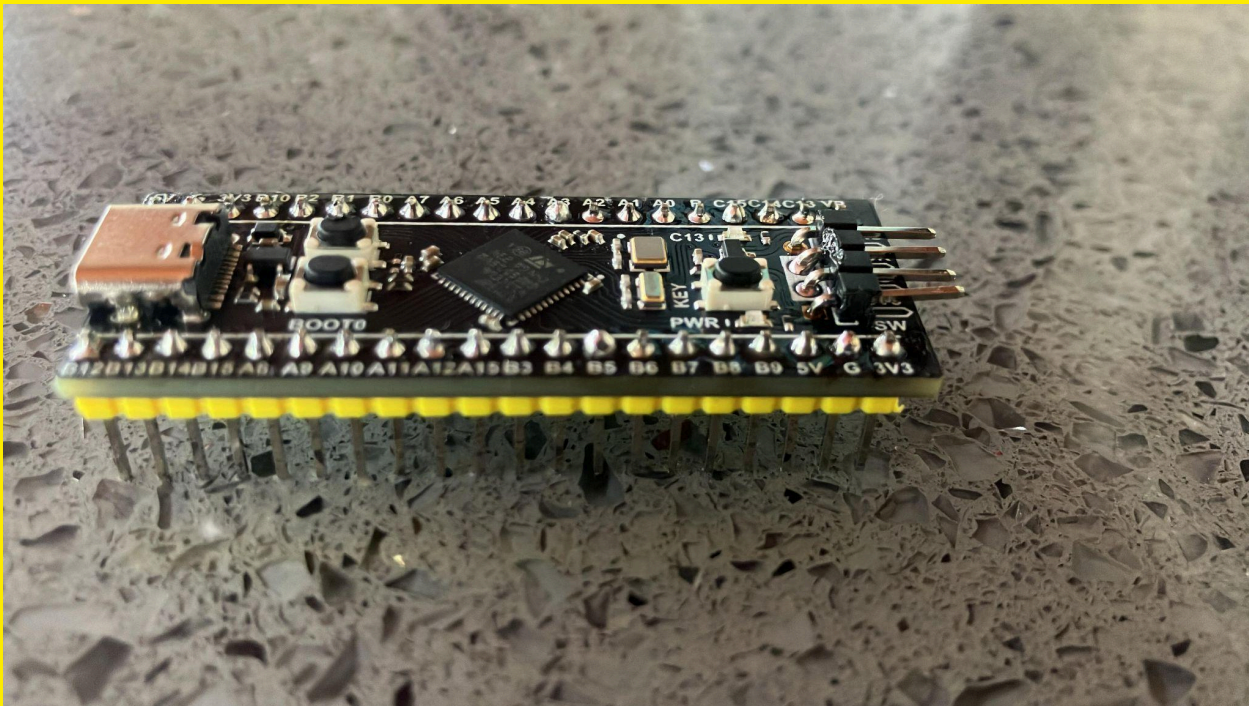
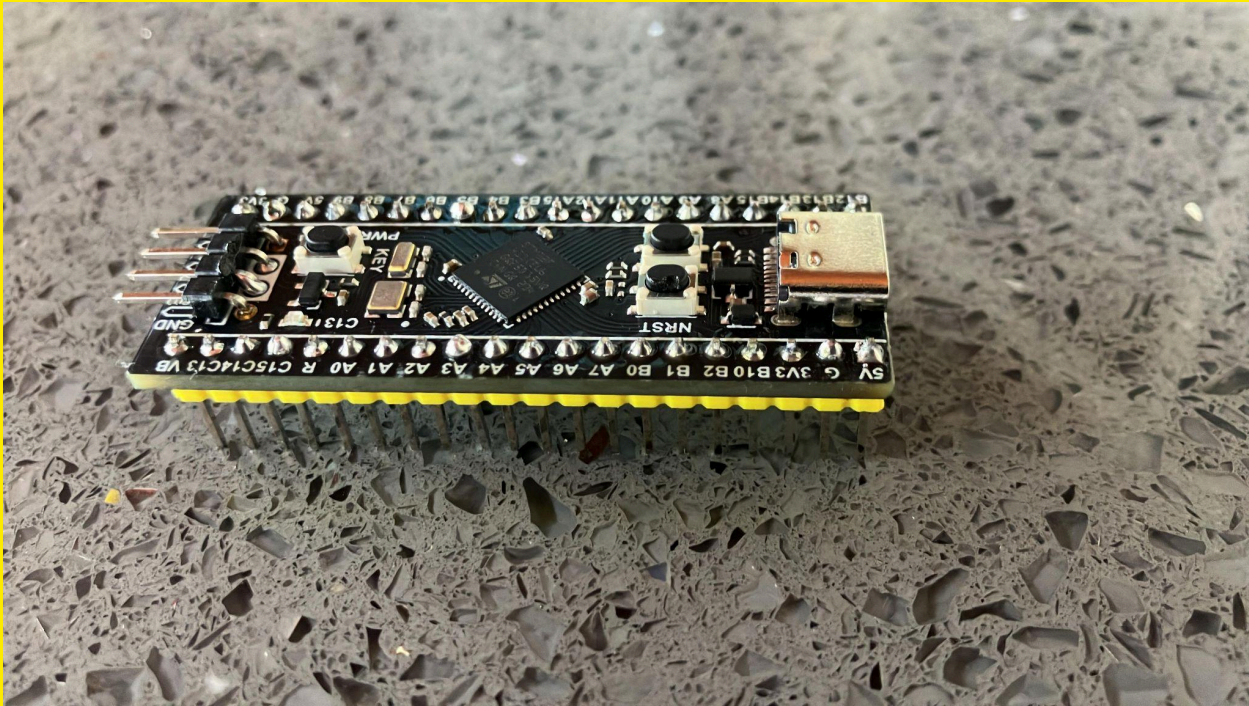
Once the parts came in, I unboxed them and made sure everything was what I ordered and checked for wrong models. Afterward, I went to my local MicroCenter to buy a simple soldering station and some solder so I can start the first step of the hardware process, which is soldering.

Before this project I had never soldered before, so it was going to be a

first time for me. I watched a couple of YouTube tutorials until I felt I was ready to start soldering. Then I ran into a problem.

-The Problem-

No matter how hot I dialed my soldering iron, it just wouldn't melt the solder. I tried a bunch of different strategies to melt the solder, but I couldn't get it to melt, or it would be very inefficient. Soldering the way I was wasn't going to cut it, and I even burned myself a couple of times, but after trying many things to get it to melt better, including swapping tips, I found the culprit of the problem. As I was substituting different pieces of the process, I decided to swap the solder as a last-ditch effort, and surprisingly it worked! The solder I had bought originally happened to be lead-free, so I hadn't realized that lead-free solder melts at a higher temperature, so it made it very inefficient to solder with it. Once that problem was solved, it was a smooth road after, and it became way easier to solder, and I even learned a couple of tricks from some videos, like how to clean the soldering iron with a wet sponge or how to make sure the solder melts the perfect shape around the pin. Finally, the finished product turned out to be pretty impressive, and I didn't even know I was capable of soldering at all, and I was pretty proud of myself in the end.



Next is programming...

IMU Programming

-Protocols-

After soldering the STM32, I had gone on vacation for around 2 weeks, but when I got back, I was ready to start the programming. Like previously, before I started anything, I had to research and write. It turns out I had to do a different type of communication with the wires, unlike normal wire communication.

-UART-

UART is one of the most common and foundational hardware communication protocols. How it works is it uses two separate lines to read and write data. The lines both go from the (RX) pins and (TX) pins. It also operates without a shared clock signal, relying instead on pre-configured matching speeds known as "baud rates." I usually set my baud rates around 112500, but depending on the model, I might go lower, like 9600. In short, a baud rate is the speed at which data travels over a serial communication channel, measured in the number of signal changes per second. Those are the fundamentals of UART, but that's actually not the protocol I used.

-I2C-

For my IMU setup, I had to use the I2C protocol. I2C, short for Inter-Integrated Circuit, is a two-wire bus that lets chips on the same board talk to each other. One device acts as the controller and coordinates everything, while the others sit on the bus and respond when addressed. In my case, the STM32 is the controller and the IMU is one of the peripherals.

Standard mode runs at 100 kHz and fast mode at 400 kHz. Faster modes exist but most Arduino libraries stick to these two. The main appeal is that you only need two pins no matter how many sensors you add, and every device identifies itself by address.

As of now these are the only protocols that I have used but if I do use more I will explain them and show how they are used.

—

-Programming-

First, I had to find my device address, which would be for the LSM303 accelerometer. To find it easier and waste less time, I used an already-made I2C scanner library. This was the code and output: Code I used for the i2c scanner:

```
#include "I2CScanner.h"

#include <Wire.h>

I2CScanner scanner;

//if you use ESP8266-01 with not default SDA and SCL pins, define these 2
lines, else delete them

#define SDA_PIN PA1

#define SCL_PIN PA0

void setup()

{
```

```
//uncomment the next line if you use custom sda and scl pins for example
with ESP8266-01 (sda = 4, scl = 5)

Wire.begin(SDA_PIN, SCL_PIN);

Serial.begin(9600);

while (!Serial) {};

scanner.Init();
}

void loop()
{
    scanner.Scan();

    delay(5000);
}
```

Output was:

--- Scan started ---

I2C device found at address 0x19 !

I2C device found at address 0x1E !

I2C device found at address 0x6B !

I2C device found at address 0x77 !

--- Scan finished ---

--

Once I had found the device address, all I had to do was find the register address and write a program to print the acceleration. At first I didn't know

where to find the register address, but I found out easily that almost all chips, modules, and more will make datasheets that show the different register addresses and more about the chip itself, and it helped me find out more about the accelerometer that I didn't know, and it has way more capabilities than I expected. This was the register map:

LSM303DLHC — Register Address Map

From the LSM303DLHC datasheet, Table 17. The chip is two I²C devices in one package: an accelerometer and a magnetometer (which also carries the temperature sensor).

Accelerometer (I²C address 0x19)

Name	Type	Hex	Binary addr	Default (bin)	Notes
Reserved (do not modify)	—	00–1F	—	—	Reserved — do not write
CTRL_REG1_A	rw	20	010 0000	00000111	
CTRL_REG2_A	rw	21	010 0001	00000000	
CTRL_REG3_A	rw	22	010 0010	00000000	
CTRL_REG4_A	rw	23	010 0011	00000000	
CTRL_REG5_A	rw	24	010 0100	00000000	
CTRL_REG6_A	rw	25	010 0101	00000000	
REFERENCE_A	rw	26	010 0110	00000000	
STATUS_REG_A	r	27	010 0111	00000000	
OUT_X_L_A	r	28	010 1000	output	X low byte
OUT_X_H_A	r	29	010 1001	output	X high byte
OUT_Y_L_A	r	2A	010 1010	output	Y low byte
OUT_Y_H_A	r	2B	010 1011	output	Y high byte
OUT_Z_L_A	r	2C	010 1100	output	Z low byte
OUT_Z_H_A	r	2D	010 1101	output	Z high byte
FIFO_CTRL_REG_A	rw	2E	010 1110	00000000	
FIFO_SRC_REG_A	r	2F	010 1111	—	
INT1_CFG_A	rw	30	011 0000	00000000	
INT1_SRC_A	r	31	011 0001	00000000	
INT1_THS_A	rw	32	011 0010	00000000	
INT1_DURATION_A	rw	33	011 0011	00000000	
INT2_CFG_A	rw	34	011 0100	00000000	
INT2_SRC_A	r	35	011 0101	00000000	
INT2_THS_A	rw	36	011 0110	00000000	
INT2_DURATION_A	rw	37	011 0111	00000000	
CLICK_CFG_A	rw	38	011 1000	00000000	
CLICK_SRC_A	rw	39	011 1001	00000000	

Name	Type	Hex	Binary addr	Default (bin)	Notes
CLICK_THS_A	rw	3A	011 1010	00000000	
TIME_LIMIT_A	rw	3B	011 1011	00000000	
TIME_LATENCY_A	rw	3C	011 1100	00000000	
TIME_WINDOW_A	rw	3D	011 1101	00000000	
Reserved (do not modify)	—	3E-3F	—	—	Reserved — do not write

Magnetometer + temperature (I²C address 0x1E)

Name	Type	Hex	Binary addr	Default (bin)	Notes
CRA_REG_M	rw	00	00000000	00010000	*
CRB_REG_M	rw	01	00000001	00100000	*
MR_REG_M	rw	02	00000010	00000011	
OUT_X_H_M	r	03	00000011	output	X high byte
OUT_X_L_M	r	04	00000100	output	X low byte
OUT_Z_H_M	r	05	00000101	output	Z high byte
OUT_Z_L_M	r	06	00000110	output	Z low byte
OUT_Y_H_M	r	07	00000111	output	Y high byte
OUT_Y_L_M	r	08	00001000	output	Y low byte
SR_REG_M	r	09	00001001	00000000	
IRA_REG_M	r	0A	00001010	01001000	
IRB_REG_M	r	0B	00001011	00110100	
IRC_REG_M	r	0C	00001100	00110011	
Reserved (do not modify)	—	0D-30	—	—	Reserved — do not write
TEMP_OUT_H_M	r	31	00110001	output	temp high byte
TEMP_OUT_L_M	r	32	00110010	output	temp low byte
Reserved (do not modify)	—	33-3A	—	—	Reserved — do not write

Notes

- Type: r = read-only, rw = read/write. "Output" means the register holds live sensor data with no fixed default.
- * The magnetometer defaults for CRA_REG_M (00010000) and CRB_REG_M (00100000) came through truncated in the paste; these are the correct 8-bit values from the datasheet.
- Watch the magnetometer output order: it is X, Z, Y (not X, Y, Z like the accelerometer), and each axis is high-byte-first. A common source of scrambled readings.
- Reserved ranges must not be written the datasheet warns this can permanently damage the device.

I already had the data from the datasheet. I just had Claude format it for me so I could easily paste it into the Google Docs. If you look through the table under "Name," you'll see something that says CTRL_REG1_A. That's the power control register address. If you look at the table again you can also see an

address with the name of that says `OUT_X_L_A`. That's another register address I need for the program, and you can tell it is for the accelerometer because the “A” stands for accelerometer. Now, this was the original program I wrote for the accelerometer:

```
#include <Arduino.h>
#include <Wire.h>
#define imuSCL PB6
#define imuSDA PB7

int ADXLAddress = 0x19; //device address
#define X_Axis_Register_DATAX0 0x28 //hexadecima adress
#define X_Axis_Register_DATAX1 0x29 //hexadecima adress
#define Power_Register 0x20 //power control

int X0, X1, X_out;

void setup(){
  Wire.begin(); //wire lib
  Serial.begin(115200);
  delay(100);

  Wire.beginTransmission(ADXLAddress); //enable measurement
  Wire.write(Power_Register);
  //Wire.write(8);
  Wire.endTransmission();
}

void loop(){
  Wire.beginTransmission(ADXLAddress);

  //Asks registers for data
  Wire.write(X_Axis_Register_DATAX0);
  Wire.write(X_Axis_Register_DATAX1);

  Wire.endTransmission(); //ends transmission

  Wire.requestFrom(ADXLAddress, 2); //request the transmitted data
```

```
if (Wire.available() >= 2) {  
    X0 = Wire.read();  
    X1 = Wire.read();  
}  
  
Serial.print("X0= ");  
Serial.println(X0);  
Serial.print("X1= ");  
Serial.println(X1);  
  
}
```

-The Problem-

For the program I got inspiration from a YouTube video, but there was one problem with the code. Usually if you get an output of "-1," then that means something is not working properly or not connected, but my output just gave me constant 0s, so I was pretty confused when I saw that since it should be giving me "-1" constantly if something is not working. So I did a test to see if it would give me "-1" at all so I unplugged the IMU from the board and it did show "-1" so that it confirmed it was working. It was just giving me 0's. So after many trials and errors, my final product was this:

-The Solution-

```
const int LSM303_ACCEL = 0x19;  
  
#define CTRL_REG1_A    0x20  
#define OUT_X_L_A      0x28  
#define AUTO_INCREMENT 0x80    // MSB of sub-address enables burst reads  
  
int16_t X_out;  
float X_ms;  
  
void setup() {  
    Wire.begin();  
    Serial.begin(115200);  
    delay(100);  
}
```

```

// ODR=100Hz, normal mode, X/Y/Z enabled -> 0101 0111
Wire.beginTransaction(LSM303_ACCEL);
Wire.write(CTRL_REG1_A); // power control register
Wire.write(0x57); // sets to normal mode (power)
Wire.endTransmission();
}

void loop() {
//write request from stm to imu
Wire.beginTransaction(LSM303_ACCEL);
Wire.write(OUT_X_L_A | AUTO_INCREMENT);
Wire.endTransmission(false); // repeated start

//reading
Wire.requestFrom(LSM303_ACCEL, 2);
if (Wire.available() >= 2) {
    int X0 = Wire.read(); // stm reads what imu wrote
    int X1 = Wire.read();
    X_out = (int16_t)((X1 << 8) | X0); // combining both registers into a
readable number (X0 and X1)
    X_out = X_out >> 4; // data is 12-bit, left-justified

    X_ms = X_out / 100.0;

    Serial.print("X = ");
    Serial.println(X_ms);
}
delay(100);
}

```

One of the many problems was that I actually had it on the wrong pins, so after I changed that, I also had to change the program. I ran into a bug in my read loop. I was doing this:

cpp

```

Wire.write(0x28); // OUT_X_L_A
Wire.write(0x29); // OUT_X_H_A

```

I thought writing both register addresses would tell the sensor to hand me back both bytes, but that's not how I²C works. The first byte I send is the register pointer, and anything after that gets treated as *data to write* into that register so my second write (0x29) was being read as a value to write into 0x28, which is read-only anyway, so it just did nothing useful.

To read two registers in a row, I only send the starting address once. And on the LSM303 there's an extra catch: I have to OR the address with 0x80 to turn on sub-address auto-increment; otherwise, the pointer stays put and I just get the same byte twice instead of moving on to 0x29.

cpp

```
Wire.write(0x28 | 0x80); // auto-increment on
```

Thereafter, I got steady readings, and it surprised me how when turning the IMU towards the ground, it can see the speed of gravity. Another thing I added to the code was a simple format for the numbers. I turned the numbers into m/s doing this. It's already in the code up there, but just mentioning it. This was just a test run. I will probably use a library now that I know how this works, and I am going to work on learning some calculus so I can make the numbers more readable and in "mph" format.

—Progress—

Next Steps:

GPS Programming

-Programming-

When I started research on the GPS, I was originally scared to do it since I thought it was hard, but it turns out it's not as hard as I thought it was. The wiring was a straightforward setup, so it doesn't need much explanation, especially since I already clarified it in the IMU programming. It uses UART, so it was an effortless and simple two-wire setup. I also originally wanted to do this without a library but I realized that would be pretty hard so I did some research to see what was the best research library to use. I still didn't and still don't as of writing this know a lot of the technical terms and stuff of the GPS, so I just picked a library that sounded convincing and one that other people have used so it's easier to debug. Furthermore, I ended up using TinyGPSPlus, and I started coding. For the first program, I mostly did it without really any the tutorial. I did look at the library example, and I made a small test program, and here was the code:

```
#include <Arduino.h>
#include <Wire.h>
#include <SoftwareSerial.h>
#include <TinyGPSPlus.h>

#define RX A0
#define TX A1

SoftwareSerial GPSSerial(RX, TX);
TinyGPSPlus gps;

void setup(){
    GPSSerial.begin(9600);
```

```
}  
  
void loop(){  
  if (GPSSerial.available() > 0) {  
    gps.encode(GPSSerial.read());  
  
    if(gps.location.isUpdated()){  
      Serial.print("Latitude = ");  
      Serial.print(gps.location.lat());  
      Serial.print("    Longitude = ");  
      Serial.println(gps.location.lng());  
      delay(1000);  
    }  
  }  
}
```

-The Problem-

My first sketch compiled and uploaded fine, but the serial monitor stayed empty. `gps.location.isUpdated()` never returned true, so I never got a latitude or longitude out of it.

-What I found-

Going back through the code, I found three separate issues, and the first one alone was enough to hide everything else.

I never called `Serial.begin()`. I opened the software serial to the module with `GPSSerial.begin(9600)`, but I never opened the USB serial to my computer. Every `Serial.print` was going nowhere. Even with a perfect satellite fix, the monitor would have stayed blank.

The `delay(1000)` was destroying my data. SoftwareSerial's receive buffer is only 64 bytes. At 9600 baud the module sends roughly 960 bytes per second, so a

one-second delay meant I was dropping about 90% of it. The NMEA sentences arrived in fragments, the checksums failed, TinyGPS++ threw them out, and `isUpdated()` had nothing to report. Lesson: Never block inside a loop that's feeding a parser.

I was only reading one byte per pass. The read was inside an `if`, so each trip through `loop()` pulled a single character. Using `while` instead drains the whole buffer every pass.

-Step 1: Confirm the module is actually talking-

Before touching the parser, I wanted to know whether bytes were arriving at all, so I flashed a bare passthrough:

cpp

```
#include <SoftwareSerial.h>
SoftwareSerial GPSSerial(A0, A1); // RX, TX

void setup() {
  Serial.begin(115200);
  GPSSerial.begin(9600);
}

void loop() {
  while (GPSSerial.available()) Serial.write(GPSSerial.read());
}
```

With the monitor at 115200, I should see `$GNRMC, . . .` and `$GNGGA, . . .` lines streaming. How to read the result:

- **Nothing at all**, wiring or baud rate. The module's TX goes to A0 and its RX goes to A1; swapping those two is the classic mistake. VCC has to match my board's logic level, and grounds need to be common. It's worth trying `GPSSerial.begin(38400)` too, since some M8 boards ship at 38400 instead of 9600.

- **Garbage characters:** baud mismatch. Try 38400, 4800, 115200.
- **Clean sentences with empty fields** (\$GNRMC,,V,,,,,,,,,N*4D)
the module is fine; I just don't have a satellite fix yet.

That last case isn't a code problem. A cold start needs a clear view of the sky, so outdoors or at a window, and the first fix can take anywhere from 30 seconds to 15 minutes. Indoors it usually never locks at all. The board has an LED that starts blinking once it acquires a fix, which is a faster check than watching the monitor.

-Step 2: Corrected sketch with diagnostics-

cpp

```
#include <SoftwareSerial.h>
#include <TinyGPSPlus.h>

SoftwareSerial GPSSerial(A0, A1); // RX, TX
TinyGPSPlus gps;
unsigned long lastReport = 0;

void setup() {
  Serial.begin(115200);
  GPSSerial.begin(9600);
  Serial.println(F("Waiting for GPS..."));
}

void loop() {
  while (GPSSerial.available() > 0) {
    gps.encode(GPSSerial.read());
  }

  if (millis() - lastReport >= 2000) {
    lastReport = millis();

    if (gps.location.isValid()) {
      Serial.print(F("Lat = "));
      Serial.print(gps.location.lat(), 6);
      Serial.print(F("   Lng = "));
      Serial.print(gps.location.lng(), 6);
```

```

Serial.print(F("  Sats = "));
Serial.println(gps.satellites.value());
} else {
Serial.print(F("No fix. chars="));
Serial.print(gps.charsProcessed());
Serial.print(F(" sentences="));
Serial.print(gps.sentencesWithFix());
Serial.print(F(" failed="));
Serial.println(gps.failedChecksum());
}
}
}

```

What I changed and why:

- `millis()` gating instead of `delay()`, so the parser keeps eating while I wait between prints.
- `isValid()` instead of `isUpdated()`, so I see the last known position rather than only the instant it changes.
- `, 6` on the coordinate prints the default two decimal places, rounding my position to about a kilometer, which is useless for a cycling computer.
- Diagnostic counters, so I can tell exactly where I'm stuck instead of guessing.

Reading the counters

What I see

What it means

chars stuck at o

Wiring or baud rate, nothing is reaching the Arduino

chars climbing,
failed climbing

Baud mismatch or dropped bytes

chars climbing,
failed o, sentences
o

Everything works; I'm just waiting on satellites

I actually ended up running into another problem, but because I had implemented the diagnostics, I easily found out I just needed a clear sky, so I stood outside, and it worked perfectly. It basically got the exact tent I was standing under in my school's pinpoint, which was crazy to see how accurate it was. Overall it gave me an output of this. Output: 25°48'46.5"N 80°16'27.2"W . Now that it's done, the next step is learning how to calculate displacement and use trilateration with it, which will be tricky, but if I make it work, I can calibrate the IMU to make a very reliable speedometer.

Next Steps & Math:

The next step was to find the actual speed, since I have the coordinates all I need is to be able to find the distance between my previous and current, subtract them by each other, and get the distance. That's the core idea obviously it's a very different formula from just displacement.

$$d = R\sqrt{\Delta\phi^2 + (\cos(\phi_m) \cdot \Delta\lambda)^2}$$

This is the formula I'll need for finding the distance between two coordinates and the cool thing is that since I'm using the formula and calculating distance so quickly I can just treat earth as a flat surface with no curvature since my initial and final are so close together.

.hpp:

Before I had mostly everything in one .cpp instead of separate inertial.cpp, .hpp, etc. so now I had to make a separate .cpp and .hpp so it's cleaner. First I had to set up my .hpp and my .cpp. For this part of the documentation I'll go more in depth since I documented a lot of it and I understand it. Now since I'm doing speed I don't need the whole debugging code and I really only need the actual function from it so this is the snippet I'll need the most:

```
if (gps.location.isValid()) {  
    Serial.print(F("Lat = "));  
    Serial.print(gps.location.lat(), 6);  
    Serial.print(F("    Lng = "));  
    Serial.print(gps.location.lng(), 6);  
}
```

But that's still really for the .cpp, which I'll get into after I explain what I did in the .hpp. First I made my namespace and class, which I called GPS. Then I declare all the variables, constants, or functions I'll need. In my public I have:

```
public:  
  
    float getLong();  
  
    float getLat();  
  
    void begin();  
  
    float getSpeed(); // current Hz = 3
```

I made simple functions that I'll use a lot, like getLong() or getLat(). void begin() initializes my GPS before I do any math. And finally, getSpeed() should be self-explanatory, but I can explain the Hz. Current Hz is how fast I'm updating or checking for speed. I needed a specific Hz that wasn't too

fast but also wasn't too resource-consuming, so I settled with 3. Now onto the private, these had most of the variables/constants.

```
private:
int deltaTime = 0; //time in seconds
//current/final
float FLong = 0;
float FLat = 0;
//change
float ChangeLong = 0;
float ChangeLat = 0;
//initial
float ILong = 0;
float ILat = 0;

int R = 6378137; //radius of earth/equator
```

For my timer I used a simple deltaTime, and I have my FinalLongitude and FinalLatitude along with the initial and change so from initial->change->final. And I have my constant for the radius of the earth but more specifically the equator since this project and where I'm testing it are pretty close to the equator, aka. Miami.

Ultimately this was the final code (for now at least):

```
namespace speedometer{
// THE constructor for GPS
class GPS{
public:
    float getLong();

    float getLat();

    void begin();

    float getSpeed(); // current Hz = 3

private:
int deltaTime = 0; //time in seconds
```

```

        //current/final
        float FLong = 0;
        float FLat = 0;
        //change
        float ChangeLong = 0;
        float ChangeLat = 0;
        //initial
        float ILong = 0;
        float ILat = 0;

        int R = 6378137; //radius of earth/equator

    };
}

```

.cpp:

The main objective for the code was really just to convert a math formula into code, which isn't too hard. I started with all the includes and basic statements:

```

#include <Arduino.h>
#include <SoftwareSerial.h>
#include <TinyGPSPlus.h>
#include <gpsController.hpp>
using namespace speedometer;

static const int RXPin = A0; // wire to GPS TX
static const int TXPin = A1; // wire to GPS RX
static const uint32_t GPSBaud = 9600;

SoftwareSerial GPSSerial(RXPin, TXPin);
TinyGPSPlus GPSMod;

```

Then I needed to initialize the GPS with the function I made before in .hpp. This really just tells it to begin like the function says:

```

void GPS::begin() {

```

```

Serial.begin(115200);
GPSSerial.begin(GPSBaud);
Serial.println(F("Booting..."));
}

```

After I did that, I started writing the code for the longitude and latitude. The code checks for the GpsSerial, and if it's available, it also calls the function that gives us the longitude we need, and this is the same for latitude. If it hasn't connected to a satellite, it'll return -1.

```

float GPS::getLong(){
    if (GPSSerial.available() > 0) {
        GPSSerial.encode(GPSSerial.read());
    }

    if (GPSSerial.location.isValid()){
        return GPSSerial.location.lng();
    }
    return -1;
}

float GPS::getLat(){
    if (GPSSerial.available() > 0) {
        GPSSerial.encode(GPSSerial.read());
    }

    if (GPSSerial.location.isValid()){
        return GPSSerial.location.lat();
    }
    return -1;
}

```

Now finally, I have to do the math. First I make my final long and lat equal to the function that gives me the coordinates, getLong()/getLat(). Then I implement the formula into my code. I did have to do research for this part because I'm not that much of an expert on doing math in C++ or writing math in c++. Here's how the math turned out,

```

float latavg = ((FLat + ILat) / 2);
ChangeLat = (FLat - ILat) * cos(latavg) * R;

```

```
ChangeLong = (FLong - ILong) * R;  
  
float Displacement = sqrt(pow(ChangeLat, 2) + pow(ChangeLong, 2));
```

Next, I make my initial long and initial lat from the getLong() and getLat(). And finally, I do my displacement divided by my delta time, and I "SHOULD" get my speed. I still haven't tested the code, but the code seems logically right, but once I do test it, it'll be under the test tab. Anyway, this is the final code for the .cpp:

```
float GPS::getSpeed(){  
    deltaTime = 333/1000;  
  
    FLong = getLong();  
    FLat = getLat();  
  
    float latavg = ((FLat + ILat) / 2);  
    ChangeLat = (FLat - ILat) * cos(latavg) * R;  
    ChangeLong = (FLong - ILong) * R;  
  
    float Displacement = sqrt(pow(ChangeLat, 2) + pow(ChangeLong, 2));  
  
    ILong = getLong();  
    ILat = getLat();  
  
    return Displacement/deltaTime;  
}
```

All I had to do was put what I made into a serial.print in main.cpp.

```
void setup(void)  
{  
    Serial.begin(9600);  
    Wire.begin(SDA_PIN, SCL_PIN); // Important do not remove connects to IMU  
  
    gyroscope.begin();  
    inertial.begin();  
    gps.begin();
```

```

    inertial.initAccel();
    gyroscope.initGyro();

}

int timer = 0;
double speed = 0;

void loop(){
    /*Serial.print("X: "); Serial.print(inertial.getX());
    Serial.print("  Y: "); Serial.print(inertial.getY());
    Serial.print("  Z: "); Serial.print(inertial.getZ());

    gyroscope.gyroTimer();
    Serial.print("  Heading: "); Serial.print(gyroscope.getHeading());
    Serial.print("  Pitch: "); Serial.print(gyroscope.getPitch());
    Serial.print("  Roll: "); Serial.println(gyroscope.getRoll());*/

    Serial.print("  Lat = "); Serial.println(gps.getLat(), 6);
    Serial.print("  Long = "); Serial.print(gps.getLong(), 6);

    /*if(timer % 333 == 0 || timer == 0){
        speed = gps.getSpeed();
        Serial.print("  Speed = "); Serial.println(gps.getSpeed(), 6);
    }

    timer += 1;
    delay(1);
    */
}

```

Final Thoughts

After getting the GPS speed calculations to work and testing it many times, I had unfortunately decided to end it there since this was really just a personal coding challenge and I still have more plans in the future for other projects.

What I learned:

- **Soldering:** Before any of this I had no knowledge of soldering or even any microcontroller coding/engineering at all, so the fact that I got to experience soldering and learn from it was a **HUGE** step for me.
- **Wire Communication:** Likewise, I had no idea that there were different protocols for wire communication, and I2C is such an interesting topic to learn, and I encourage anybody learning coding microcontrollers for their first time to watch videos on it because it's crazy what

it can do, and I'll definitely be using it in the future.

- **Basic Programming: While coding, I learned how to implement formulas into code, which made my life way easier, and the use of PlatformIO helped me a lot. I definitely earned some knowledge from making this speedometer.**
- **Limits: Finally, I learned that things seem harder than they are. Coding a speedometer from scratch wasn't as easy as I thought it would be, and it definitely humbled me a bit. Now, because of this, I learned what my limits are and the difficulty differences in other projects.**

Thoughts:

Even though I know this coding challenge is probably easy for other people, I struggled a bit, and all the times I struggled I was learning, so this project definitely completed the main goal of helping me learn more in the process. Even though I never completed this project fully, it doesn't mean this project is over. My new goal is to potentially finish this in the future when I have more knowledge on this subject. And the fact I was able to use this without AI is another goal I accomplished.

In the Future:

Like I said earlier, I wish to come back to this and finish this project, but using the skills I learned, I'll definitely be able to make a better project. I'm proud of what I accomplished since this was my first project that I started in 7th grade. This is only the start of many projects to come, and I'm glad I got to learn from this.
